MATH 231 – Linear Algebra I

A Natural Progression Through the Course Material

Fall 2026 – working draft

Draft dated July 1, 2026

How to read this document

The tentative syllabus contains *more than one semester* of material by design: the intent is a wide breadth from which to select the topics students find most compelling and the instructor deems most essential. Accordingly:

- Units are ordered so that every concept is preceded by its prerequisites. The **Domain** tag names the discipline; the *objects* name the primary mathematical objects in play.
- Priority tags let you carve a one-semester path: [Core] = spine of the course (skip nothing); [Recommended] = strongly desired if time permits; [Enrichment] = optional depth / seminar-style topics.
- Each unit ends with an explicit **Gateway**: the competencies a student must hold before the next unit is intelligible.
- The course is grouped into four *arcs*. A minimal one-semester realization is Arcs I–III plus a chosen slice of Arc IV.

Domains used: Pure Linear Algebra; Numerical Analysis / Numerical Linear Algebra; Multivariable Calculus; Optimization; Machine Learning; Spectral Graph Theory.

Implementation languages: C++ (low-level kernels), Python (high-level demos), MATLAB (verification).

Textbook reference keys (each unit lists chapter/lecture-level pointers, not page-exact): **[BV]** Boyd & Vandenberghe, *Introduction to Applied Linear Algebra*; **[TB]** Trefethen & Bau, *Numerical Linear Algebra* (cited by *Lecture*); **[S]** Strang, *Linear Algebra and Its Applications* (4th ed.); **[A]** Axler, *Linear Algebra Done Right* (4th ed.). Topics lying outside all four texts are marked *supplementary*.

Dependency map (arc-level)

Arc	Units	Depends on
I. Foundations	1–3	—
II. Matrices & Continuous Math	4–6	Arc I
III. Core Matrix Theory	7–9	Arcs I–II
IV. Advanced Methods & Apps	10–12	Arcs I–III

Two "honest" forward references worth naming up front: the *spectral* and *nuclear* matrix norms are *defined* in Unit 4 but only *computed/justified* after the SVD in Unit 11; the *Hessian* is introduced in Unit 5 but its definiteness analysis is completed in Unit 11.

Arc I – Foundations

1. Vectors, Vector Spaces & the Language of the Course

Domain(s)
Pure Linear Algebra [Core]

Primary object(s)
sets, fields $(\mathbb{R}, \mathbb{C})$*, vectors*

Topics
Sets and functions; the field axioms for $\mathbb{R}$ and $\mathbb{C}$. Vectors in $\mathbb{R}^2, \mathbb{R}^3, \mathbb{R}^n$ and higher-dimensional spaces. The algebraic properties: vector addition, scalar multiplication, additive identity/inverse, commutativity, associativity, the zero vector, distributivity, multiplicative identity. **Formal definition of a vector space** (a set V with $+$ and scalar multiplication satisfying the six axioms). Subspaces; span; linear independence/dependence; basis; dimension. Geometric picture: parallelogram rule, scaling, normalization.

Key theorems
Uniqueness of the additive identity and additive inverse; every spanning set contains a basis / every independent set extends to a basis [Recommended]; dimension is well-defined (basis-size invariance) [Recommended].

Tools / formulas
Linear combinations; coordinate representation relative to a basis.

Algorithms
— (algorithmic content begins in Unit 2).

Applications
Image vectorization (an image as a point in $\mathbb{R}^{HW}$); physical vectors (position/velocity/acceleration) as a motivating instance [Recommended].

Languages
Python (construct/visualize vectors), C++ [Enrichment].

Textbook refs
[A] Ch. 1–2 (vector spaces, independence, bases, dimension); **[S]** Ch. 2; **[BV]** Ch. 1, 5.

ENTER
High-school algebra; comfort with set notation.

Gateway to next unit. Student can state the six vector-space axioms, verify whether a given set with operations is a vector space, and compute a basis/dimension of a subspace of $\mathbb{R}^n$.

2. Inner Products, Norms & Similarity

Domain(s)
Pure Linear Algebra + **Numerical LA** + **Machine Learning** [Core]

Primary object(s)
vectors

Topics
Dot (inner) product; component and orthogonal projection of one vector onto another; angle between vectors. Orthogonal vs. parallel vectors. **Vector norms**: Euclidean/standard (ℓ_2), Manhattan/taxicab (ℓ_1), Chebyshev/chessboard (ℓ_∞), and the general p-norm. Normalization. Distance/metric induced by a norm. Cosine similarity and general similarity kernels.

Key theorems
Cauchy–Schwarz inequality; triangle inequality; Pythagorean identity for orthogonal vectors [Recommended].

Tools / formulas
Dot product $\langle x, y\rangle$; projection $\text{proj}_y\, x = \frac{\langle x,y\rangle}{\langle y,y\rangle} y$; cosine similarity $\frac{\langle x,y\rangle}{\|x\|\,\|y\|}$; p-norm $\|x\|_p = (\sum |x_i|^p)^{1/p}$; general kernel $k(x, y)$.

Algorithms
Gram–Schmidt orthogonalization (classical form; modified form deferred to Unit 10) [Core]; **k-Nearest Neighbors** [Recommended]; **k-means clustering** [Recommended].

Applications
Semantic embeddings in language models (cosine similarity between token/word vectors); retrieval; clustering; a geometric preview of the SVM margin [Enrichment].

Languages
C++ kernels (inner product, cosine similarity, with compiler-flag optimization) [Core]; Python demos (KNN, k-means) [Recommended]; MATLAB verification.

Textbook refs
[**A**] Ch. 6 (inner product spaces); [**TB**] Lec. 2–3 (orthogonal vectors, norms), Lec. 8 (Gram–Schmidt); [**S**] Ch. 3; [**BV**] Ch. 3–5.

ENTER
Unit 1 (vectors, span, basis).

Gateway to next unit. Student can compute norms/projections/cosine similarity by hand and in code, prove Cauchy–Schwarz, and run Gram–Schmidt to produce an orthonormal set.

3. Complex Numbers & Quaternions

Domain(s)
Pure Linear Algebra / Algebra [Core (complex)], [Enrichment (quaternions)]

Primary object(s)
complex numbers $\mathbb{C}$, *quaternions* $\mathbb{H}$

Topics
Algebraic structure of $\mathbb{C}$: addition, multiplication, conjugation. Real and imaginary parts; the modulus. Polar form; the complex plane and the geometric interpretation of multiplication as rotation-and-scaling. Quaternions: non-commutative multiplication, use in 3-D rotation [Enrichment].

Key theorems
Euler's formula $e^{i\theta} = \cos\theta + i\sin\theta$; **De Moivre's theorem**; Fundamental Theorem of Algebra (stated) [Recommended].

Tools / formulas
Modulus $|z| = \sqrt{a^2+b^2}$; $\operatorname{Re} z, \operatorname{Im} z$; polar form $z = re^{i\theta}$; n-th roots of unity [Recommended]; quaternion product / rotation formula [Enrichment].

Algorithms
Complex arithmetic; rotation composition via quaternions [Enrichment].

Applications
Signal representation (phasors) [Recommended]; 3-D rotations in graphics and robotics [Enrichment]; foreshadowing complex eigenvalues (Unit 11).

Languages
Python (complex plane visualization); MATLAB.

Textbook refs
[A] Ch. 1 ($\mathbb{F} = \mathbb{R}$ or $\mathbb{C}$), Ch. 4 (polynomials, fundamental theorem of algebra); Euler's formula, De Moivre, quaternions *supplementary*.

ENTER
Unit 1 (fields, vectors); Unit 2 (norms $\Rightarrow$ modulus).

Gateway to next unit. Student can move fluently between rectangular and polar form, apply Euler/De Moivre, and interpret complex multiplication geometrically.

Arc II – Matrices & Continuous Math

4. Matrices: Algebra, Determinant, Trace & Matrix Norms

Domain(s)
Pure Linear Algebra + **Numerical LA** [Core]

Primary object(s)
matrices

Topics
Matrices as arrays and as objects: addition, scalar multiplication, matrix multiplication, transpose/conjugate-transpose, inverse. Special matrices (identity, diagonal, symmetric, orthogonal). Determinant and its properties; trace. The matrix exponential e^A (defined by its series; closed forms deferred to Unit 11). **Matrix norms**: induced/operator 1-norm, ∞-norm, spectral (2-)norm; Frobenius norm; nuclear norm.

Key theorems
Determinant multiplicativity $\det(AB) = \det A \det B$; invertibility $\iff \det \neq 0$; submultiplicativity of induced norms $\|AB\| \leq \|A\|\|B\|$ [Recommended].

Tools / formulas
$\operatorname{tr}(A) = \sum a_{ii}$; determinant (cofactor and via elimination); $\|A\|_F = \sqrt{\sum a_{ij}^2}$; induced norm $\|A\| = \sup_{x \neq 0} \|Ax\|/\|x\|$; $e^A = \sum_k A^k/k!$. *Note:* spectral and nuclear norms are *defined* here but *computed* via the SVD in Unit 11.

Algorithms
Naive and blocked matrix multiply; matrix–vector product [Core].

Applications
Linear operators on data; adjacency/weight matrices (preview of graphs).

Languages
C++ (matmul, mat–vec with `-O2/-O3`, cache blocking) [Core]; Python/MATLAB verification.

Textbook refs
[S] Ch. 1 (matrix operations), Ch. 4 (determinants); **[A]** Ch. 3, Ch. 9 (trace, determinant); **[TB]** Lec. 1, 3; **[BV]** Ch. 6, 10–11; matrix exponential *supplementary.*

ENTER
Units 1–2 (vectors, norms).

Gateway to next unit. Student can multiply/invert matrices, compute determinant and trace, and evaluate the Frobenius and induced $1/\infty$ norms; understands what the spectral/nuclear norms *mean* even before computing them.

5. Multivariable Calculus: Gradients, Hessians & Taylor's Theorem

Domain(s)
Multivariable Calculus [Core]

Primary object(s)
multivariate functions, vectors, matrices

Topics
Multivariable scalar functions and vector-valued functions; their graphs. Equations of planes; tangent planes. The definition of the derivative for multivariate scalar functions; partial derivatives. Linear approximation. **The gradient** (a vector) and **the Hessian** (a matrix; definiteness analysis completed in Unit 11). Review of exponential and logarithmic functions and *derivations* of the derivatives of e^x and $\ln x$.

Key theorems
Taylor's theorem, univariate and multivariate (first- and second-order forms) [Core]; equality of mixed partials (Clairaut) [Recommended].

Tools / formulas
Gradient ∇f; Hessian H_f; tangent-plane / first-order model $f(x) \approx f(a) + \nabla f(a)^\top (x-a)$; second-order model with $\frac{1}{2}(x-a)^\top H (x-a)$; $\frac{d}{dx} e^x = e^x$, $\frac{d}{dx} \ln x = 1/x$.

Algorithms
Numerical gradient (finite differences) [Recommended].

Applications
Sets the analytic foundation for all of Arc IV (optimization, regression).

Languages
Python (surface/contour plots, gradient fields); MATLAB.

Textbook refs
[**BV**] App. C (derivatives & optimization); Taylor's theorem and the Hessian *supplementary* (multivariable-calculus text).

ENTER
Units 1–2 (vectors, inner products) and Unit 4 (matrices, for the Hessian).

Gateway to next unit. Student can compute gradients/Hessians, write first- and second-order Taylor models of a multivariate function, and derive $\frac{d}{dx}e^x$ and $\frac{d}{dx}\ln x$ from first principles.

6. Numerical Foundations: Complexity, Floating Point, Conditioning & Stability

Domain(s)
Numerical Analysis [Core (short unit)]

Primary object(s)
real numbers, floating-point numbers, functions

Topics
Time complexity: Big-O, Ω, Θ. The floating-point number system as the footing of numerical analysis. The **condition number** of a (univariate, then matrix) problem $\Rightarrow$ sensitivity of outputs to input perturbation. **Stability** of algorithms. Sources of numerical error: cancellation, rounding, precision. Tolerances and stopping criteria in iterative algorithms.

Key theorems
Relationship error $\lesssim$ (condition number) $\times$ (machine epsilon) for stable algorithms [Recommended].

Tools / formulas
Machine epsilon; condition number $\kappa(f) = |xf'(x)/f(x)|$ (univariate) and $\kappa(A) = \|A\|\,\|A^{-1}\|$ (matrix, referenced forward to Unit 11); absolute vs. relative error.

Algorithms
Illustrative unstable vs. stable computations (e.g., catastrophic cancellation demos) [Core].

Applications
Governs the reliability of every later algorithm (elimination, QR, eigen).

Languages
C++ (float vs. double behavior); Python/MATLAB (epsilon, ill-conditioned examples).

Textbook refs
[**TB**] Lec. 12–15 (conditioning, floating point, stability); Big-Oh / complexity *supplementary*.

ENTER
Unit 4 (matrix norms); Unit 5 (derivatives, for condition numbers).

Gateway to next unit. Student can give asymptotic costs, explain what conditioning and stability mean, identify cancellation, and set a sensible tolerance for an iterative method.

Arc III – Core Matrix Theory

7. Linear Systems & Elimination

Domain(s)
Pure Linear Algebra + **Numerical LA** [Core]

Primary object(s)
matrices, vectors

Topics
Systems $Ax = b$; row operations; row-echelon and reduced row-echelon form. Existence and uniqueness of solutions; rank; pivots; homogeneous vs. inhomogeneous systems. **LU factorization**; pivoting for stability (tie to Unit 6).

Key theorems
Rouché–Capelli (solvability via rank) [Recommended]; existence/uniqueness of LU with partial pivoting [Recommended].

Tools / formulas
Gaussian/Gauss–Jordan elimination; back/forward substitution; $A = LU$, $PA = LU$.

Algorithms
Gaussian elimination; **LU factorization** with partial pivoting [Core].

Applications
The computational workhorse behind regression, Newton steps, and more.

Languages
C++ (elimination kernel) [Recommended]; MATLAB (`lu`, verification) [Core].

Textbook refs
[S] Ch. 1 (Gaussian elimination); **[TB]** Lec. 20–22 (elimination, pivoting, stability); **[BV]** Ch. 8, 11.

ENTER
Units 1–2, 4 (vectors, matrices); Unit 6 (why pivoting).

Gateway to next unit. Student can solve $Ax = b$ by elimination, determine solvability from rank, and produce/use an LU factorization.

8. Linear Transformations

Domain(s)
Pure Linear Algebra [Core]

Primary object(s)
linear maps, matrices

Topics
Linear maps $T : V \to W$; the matrix of a linear map; composition as matrix product. Kernel (null space) and image (range) of a map. Injectivity/ surjectivity/invertibility. Change of basis and similarity.

Key theorems
Rank–Nullity theorem; a linear map is invertible $\iff$ its matrix is invertible [Core]; change-of-basis / similarity relation $B = P^{-1}AP$ [Recommended].

Tools / formulas
Matrix representation in a basis; change-of-basis matrix P.

Algorithms
Computing kernel/image bases via elimination [Core].

Applications
Rotations/reflections/projections as maps; graphics transforms [Recommended]; sets up the four subspaces and eigen-theory.

Languages
Python (visualize 2-D/3-D transforms); MATLAB.

Textbook refs
[A] Ch. 3 (linear maps); **[S]** App. A (linear transformations); **[BV]** Ch. 2 (linear functions).

ENTER
Unit 7 (elimination, rank); Units 1–2.

Gateway to next unit. Student can pass between a linear map and its matrix, compute kernel/image, apply rank–nullity, and change basis.

9. The Four Fundamental Subspaces

Domain(s)
Pure Linear Algebra [Core]

Primary object(s)
subspaces, matrices

Topics
Column space, null space, row space, left null space of A; their dimensions and how they follow from rank. Orthogonality relations between the pairs. The geometry that underlies projection and least squares.

Key theorems
Fundamental Theorem of Linear Algebra (dimensions + orthogonality: $\mathcal{N}(A) \perp \mathcal{R}(A^\top)$, $\mathcal{N}(A^\top) \perp \mathcal{R}(A)$) [Core].

Tools / formulas
$\dim \mathcal{C}(A) = \operatorname{rank} A = r$; $\dim \mathcal{N}(A) = n - r$; bases of each subspace from RREF.

Algorithms
Extract bases of all four subspaces from elimination [Core].

Applications
Direct foundation for least squares (Unit 10) and PCA/SVD geometry (Unit 11).

Languages
Python/MATLAB (rank, null-space bases).

Textbook refs
[S] Ch. 2 (§2.4, the four subspaces); **[A]** Ch. 3 (null space & range, fundamental theorem of linear maps); **[BV]** Ch. 5.

ENTER
Units 7–8 (elimination, maps, rank–nullity).

Gateway to next unit. Student can name and produce bases for all four subspaces of a given A and state their dimensions and orthogonality relations.

Arc IV – Advanced Methods & Applications

10. Orthogonality, Projections, Least Squares & Factorizations

Domain(s)
Numerical LA + Machine Learning [Core]

Primary object(s)
matrices, vectors, subspaces

Topics
Orthogonal projection onto a subspace; orthonormal bases. The least-squares problem $\min_x \|Ax - b\|_2$ and the normal equations $A^\top Ax = A^\top b$. Linear regression as least squares. **QR factorization** (classical and *modified* Gram–Schmidt; Householder reflections). **Cholesky factorization** of the SPD matrix $A^\top A$. Introduction to quadratic forms (definiteness deepened in Unit 11). Orthogonal transforms as a change to an orthonormal basis; the **Discrete Cosine Transform (DCT)** as a *fixed* orthonormal basis, contrasted with the data-adaptive SVD basis (payoff in the JPEG/SVD image-compression demo, Unit 11) [Recommended].

Key theorems
Projection/orthogonality principle (residual $\perp$ column space); existence & uniqueness of the least-squares solution when A has full column rank [Core]; $A = QR$ existence [Recommended].

Tools / formulas
Projection matrix $P = A(A^\top A)^{-1}A^\top$; normal equations; $A = QR$; $A^\top A = LL^\top$ (Cholesky); quadratic form $q(x) = x^\top Ax$.

Algorithms
Ordinary Least Squares (OLS) [Core]; **Modified Gram–Schmidt** [Core]; **Householder QR** [Recommended]; **Cholesky factorization** [Recommended].

Applications
Linear regression / curve fitting [Core]; data whitening [Enrichment].

Languages
Python (linear regression demo) [Core]; MATLAB (`qr`, `chol`, verification); C++ (QR kernel) [Enrichment].

Textbook refs
[**TB**] Lec. 6–11 (projectors, QR, Gram–Schmidt, Householder, least squares), Lec. 23 (Cholesky); [**BV**] Ch. 12–13; [**S**] Ch. 3, Ch. 6; [**A**] Ch. 6.

ENTER
Unit 9 (subspaces, projection geometry); Unit 6 (why MGS/Householder over normal equations).

Gateway to next unit. Student can set up and solve a least-squares problem three ways (normal equations, QR, Cholesky), fit a regression model, and explain why QR is more stable than the normal equations.

11. Eigenvalues, Eigenvectors & the Singular Value Decomposition

Domain(s)
Pure LA + **Numerical LA** + **Spectral Graph Theory** + **Machine Learning** [Core]

Primary object(s)
matrices, vectors, complex numbers

Topics
Eigenvalues/eigenvectors; characteristic polynomial; connection to determinant ($\prod \lambda_i$) and trace ($\sum \lambda_i$). Diagonalization and similarity. Symmetric matrices and orthogonal diagonalization. **Quadratic forms and definiteness** (completes Unit 10). The **SVD** and its geometry; low-rank approximation; *now* the spectral and nuclear norms of Unit 4 are computable. Why eigenvalue problems are solved *iteratively*. Graph Laplacians and spectral clustering. Stochastic (Markov) matrices and the dominant-eigenvector interpretation that underlies PageRank.

Key theorems
Spectral theorem (symmetric/Hermitian $\Rightarrow$ orthonormal eigenbasis) [Core]; **Schur's theorem** (unitary triangularization) [Recommended]; **Singular Value Decomposition** [Core]; **Eckart–Young** (best low-rank approximation) [Recommended]; **Abel–Ruffini theorem** (no general radical solution for degree ≥ 5 $\Rightarrow$ eigenvalues need iteration) [Enrichment]; **Perron–Frobenius theorem** (a non-negative/stochastic matrix has a positive dominant eigenvalue with a nonnegative eigenvector; underlies PageRank and spectral clustering) [Recommended].

Tools / formulas
Characteristic polynomial $\det(A - \lambda I) = 0$; $A = Q\Lambda Q^\top$; $A = U\Sigma V^\top$; $\|A\|_2 = \sigma_{\max}$, $\|A\|_* = \sum \sigma_i$; $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$; matrix exponential via eigendecomposition $e^A = Qe^\Lambda Q^{-1}$; graph Laplacian $L = D - W$.

Algorithms
Power iteration; QR algorithm (stated) [Recommended]; **PCA** [Core]; **Randomized SVD** and **Hutchinson trace estimation** (*Monte-Carlo application*) [Enrichment]; spectral clustering on graph Laplacians [Recommended].

Applications
PCA / dimensionality reduction; Eigenfaces / "Eigencats" [Recommended]; spectral graph theory and community detection [Recommended]; low-rank compression [Enrichment]; Latent Semantic Analysis on a TF–IDF term–document matrix (document retrieval); PageRank; SVD/DCT image compression; static word-embedding geometry (see Projects appendix).

Languages
Python (PCA, SVD, Eigenfaces, spectral graph demos) [Core]; MATLAB (`eig`, `svd`, verification); C++ (power iteration) [Enrichment].

Textbook refs
[A] Ch. 5, Ch. 7 (spectral theorem, SVD), Ch. 8 (Schur); **[S]** Ch. 5–6; **[TB]** Lec. 4–5 (SVD), Lec. 24–31 (eigenvalue algorithms, computing the SVD); PageRank, Perron–Frobenius, spectral graph theory *supplementary*.

ENTER
Units 8–10 (maps, subspaces, projections, quadratic forms); Unit 3 (complex eigenvalues); Unit 6 (why iterative).

Gateway to next unit. Student can diagonalize symmetric matrices, compute/interpret an SVD, use it for low-rank approximation and PCA, evaluate the spectral/nuclear norms and κ_2, and build a graph Laplacian for clustering.

12. Numerical Optimization & Machine-Learning Capstone

Domain(s)
Optimization + **Machine Learning** [Core (grad. descent, Newton); Recommended/Enrichment beyond]

Primary object(s)
multivariate functions, matrices, vectors

Topics
Unconstrained minimization; descent directions; step sizes/line search; convergence and its dependence on conditioning (tie to Units 6,11). **Gradient descent**; **Newton's method** (Hessian solve = a linear system, Unit 7). Nonlinear least squares: **Gauss–Newton** and **Levenberg–Marquardt**. Quasi-Newton **BFGS** (mention). Constrained optimization and the **support vector machine** under the Vapnik objective; kernel methods (reuse Unit 2 kernels). Stochastic / Monte-Carlo optimization (SGD) as an application [Enrichment]. Low-rank matrix factorization with a *masked* loss over observed entries (Alternating Least Squares / SGD) for collaborative filtering. Model-selection methodology: train/validation/test splits and cross-validation as estimates of generalization error (ties to Unit 6 tolerances) [Recommended].

Key theorems
First-/second-order optimality conditions (via Taylor, Unit 5); convexity ⇒ global optimum [Recommended]; local quadratic convergence of Newton's method [Enrichment].

Tools / formulas
GD update $x_{k+1} = x_k - \alpha \nabla f$; Newton $x_{k+1} = x_k - H^{-1}\nabla f$; Gauss–Newton normal-equation step; LM damped step $(J^\top J + \lambda I)\Delta = -J^\top r$; SVM margin/objective.

Algorithms

Gradient descent [Core]; **Newton's method** [Core]; **Gauss–Newton** [Recommended]; **Levenberg–Marquardt** [Recommended]; **BFGS** (mention) [Enrichment]; **SVM training** [Recommended]; **SGD / Monte-Carlo** [Enrichment].

Applications

Model fitting; classification via SVM; the full ML pipeline tying the course together (embeddings → similarity → optimization); collaborative- filtering recommender; signal trilateration (GPS-style positioning).

Languages

Python (GD, Newton, SVM demos) [Core]; MATLAB (`lsqnonlin`, verification) [Recommended]; C++ (GD kernel) [Enrichment].

Textbook refs

[BV] Ch. 18–19 (nonlinear & constrained least squares: Gauss–Newton, Levenberg–Marquardt), App. C; gradient descent, Newton, BFGS, SVM *supplementary*.

ENTER

Unit 5 (gradient/Hessian/Taylor); Unit 7 (linear solves); Unit 10 (least squares); Unit 11 (definiteness/conditioning).

Gateway to next unit. Course capstone: student can derive and implement gradient descent and Newton's method, choose Gauss–Newton/LM for nonlinear least squares, and train an SVM, articulating how each method's behavior depends on conditioning.

Assessment mapping (tentative)

Assessment	Units covered	Suggested focus
Exam 1	1–6	vector spaces, norms, complex, matrices,
Exam 2	7–10	systems, maps, subspaces, least squares
Final Exam (cheat sheet)	1–12 (weighted to 11–12)	eigen/SVD, optimization, synthesis
Project 1 [Assigned]	after 2	#1 K-means image color segmentation (
Project 2 [Assigned]	after 11	#5 Eigencats via PCA (Python)
Project 3 [Assigned]	after 12	#7 Mini-ML: OLS + SVM + KNN with
Project 4 [Assigned]	after 12	#8 Signal trilateration: Gauss–Newton /
Lecture demos	10–12	#2 SVD+DCT, #3 LSA, #4 collab. filte
Homework 1–10	one per unit (Units 1–12, some combined)	mixed hand + MATLAB verification

Computational Projects & Application Demos

This is a *menu*, not an assignment quota: every item below is documented, but only a small subset is assigned to students as an implementation ([**Assigned**]); the complement are instructor-led [**Demo**]s that still put the application in front of the class. The list is open — further items may be appended as #10, #11, … without disturbing the ordering.

Outline additions these items introduced: DCT / orthogonal transforms (Unit 10); Perron–Frobenius theorem and stochastic matrices (Unit 11); TF–IDF term–document weighting (Unit 11); masked-loss / ALS factorization and train/validation/test model selection (Unit 12).

#	Project	Role	Prereq	LA machinery
1	K-means color segmentation	Assigned	U2	K-means, image vectorization, ℓ_2 distance
2	SVD + DCT image compression	Demo	U11	SVD, Eckart–Young, DCT orthonormal basis
3	LSA file-system search	Demo	U11	truncated SVD, TF–IDF, cosine similarity
4	Collaborative filtering	Demo	U12	masked low-rank factorization, ALS/SGD
5	Eigencats (PCA)	Assigned	U11	PCA, covariance eigenbasis / SVD
6	PageRank	Demo	U11	power method, stochastic matrix, Perron–Frobenius
7	Mini-ML (OLS/SVM/KNN)	Assigned	U12	OLS, SVM, KNN, train/val/test
8	Signal trilateration	Assigned	U12	Gauss–Newton, Levenberg–Marquardt, Jacobian
9	Semantic embedding geometry	Demo	U11	cosine similarity, KNN, PCA, vector arithmetic

Assigned implementations

#1 – K-means image color segmentation (after Unit 2)

Cluster an image's pixels in RGB (or CIELAB) space into k colors and replace each pixel by its cluster centroid; display the quantized/segmented image as a function of k. *LA focus:* ℓ_2 distance, centroids as means, Lloyd's iteration and its convergence. Python (NumPy/Pillow) or a C++ kernel.

#5 – Eigencats via PCA (after Unit 11)

Assemble a data matrix of aligned cat (or face) images, mean-center, and compute principal components via the SVD; visualize the top "eigencats," reconstruct images from r components, and plot reconstruction error and variance-explained vs. r. *LA focus:* covariance eigenstructure, SVD, low-rank reconstruction. Python.

#7 – Mini-ML with validation (after Unit 12)

Three sub-tasks on suitable datasets: OLS for a regression problem, an SVM for a classification problem, and KNN for a multiclass problem; use a train/validation/test split, report generalization metrics, and justify model choice. *LA focus:* normal equations vs. QR (OLS), margin/kernels (SVM), distance metrics (KNN), validation methodology. Python (hand-implement the cores; library baselines allowed).

#8 – Signal trilateration (after Unit 12)

From noisy range measurements to known anchors, estimate an emitter's position by nonlinear least squares; implement Gauss–Newton and Levenberg–Marquardt and compare convergence/robustness vs. initialization and noise. *LA focus:* residual Jacobian, GN normal-equation step, LM damping, conditioning. Python or MATLAB.

Lecture demos

#2 – SVD + DCT image compression (Unit 11)

Sweep the rank r of an image's SVD and watch quality vs. storage; then show the 8×8 block DCT that JPEG actually uses, connecting a *fixed* orthonormal basis to the data-adaptive SVD basis. Motivates Eckart–Young.

#3 – Latent Semantic Analysis search (Unit 11)

Build a small term–document matrix over a mock file system, TF–IDF weight it, truncate its SVD, and answer queries by cosine similarity in the reduced "concept" space; contrast with exact keyword matching.

#4 – Collaborative filtering (Unit 12)

On a toy ratings matrix with missing entries, fit a low-rank factorization by ALS/SGD over *observed* entries and predict held-out ratings; frames recommendation as matrix completion.

#6 – PageRank (Unit 11)

Model a small web graph as a stochastic matrix, run the power method, and watch the ranking vector converge to the dominant eigenvector; connect to Perron–Frobenius and the damping factor.

#9 – Semantic embedding geometry (Unit 11, static)

With pretrained word vectors: nearest neighbors under cosine similarity, analogy arithmetic (king − man + woman $\approx$ queen), PCA to 2-D for visualization, and clustering — purely linear algebra, no model training.

One-semester carving suggestion

A realistic 14–15 week path keeps all [Core] items and a chosen subset of [Recommended]:

- **Arc I** (Units 1–3): $\sim$3 weeks. Quaternions $\rightarrow$ Enrichment.
- **Arc II** (Units 4–6): $\sim$3 weeks. Matrix-exp $\rightarrow$ light.
- **Arc III** (Units 7–9): $\sim$3 weeks.
- **Arc IV** (Units 10–12): $\sim$4–5 weeks. Pick *one* of {Levenberg–Marquardt, spectral graph theory, randomized SVD} as the enrichment capstone rather than all three.

Reference texts. Boyd & Vandenberghe, *Introduction to Applied Linear Algebra* (applied spine); Trefethen & Bau, *Numerical Linear Algebra* (Arc II & IV numerics); Strang, *Linear Algebra and Its Applications* (subspaces & intuition); Axler, *Linear Algebra Done Right* (theoretical backbone, Arcs I & III).